

LAB

Topic 5&6: Relational Division - Stored Procedure & Function



Lecturer: Lê Thị Nhàn, PhD.
Lab instructor: Nguyễn Ngọc Toàn, MSc.
Nguyễn Ngọc Minh Châu, MSc.

Relational Division

Query: Find all students who registered (1) for all section_ID of the Spring semester in the academic year 2022 (2).

OR

Find all students who registered (1) every section_ID of the Spring semester in the academic year 2022 (2).

Identify the Division Components:

- **Dividend (1) - what we have:** (All) student registrations => **GradeReport** table
- **Divisor (2) - what we want to cover:** (All) section of the Spring semester in the academic year 2022 => **Section** table filtered to Spring 2022

Using NOT EXISTS

```
SELECT DISTINCT x.Student_ID
FROM GradeReport AS X
WHERE NOT EXISTS (
  SELECT *
  FROM Section Y WHERE semester= 'Spring' and school_year = 2022
  AND NOT EXISTS (
    SELECT *
    FROM GradeReport AS Z
    WHERE Z.Student_ID = X.Student_ID
    AND Z.section_ID = Y.Section_ID
  )
)
```

X and **Z** are the dividend table. **Y** is the divisor.

How It Works:

- For each student x.Student_ID, we check if there is **any** section y.Section_ID from Spring 2022...
- ...for which **no corresponding row** exists in GradeReport (i.e., the student didn't register for that section).
- If such a missing registration **does not exist**, the student **must have registered for all** required sections.

Using EXCEPT

```
SELECT DISTINCT x.Student_ID
FROM GradeReport AS x
WHERE NOT EXISTS (
    SELECT Section_ID
    FROM Section y WHERE semester= 'Spring' and school_year = 2022

    EXCEPT

    SELECT s.Section_ID
    FROM GradeReport z join section s on z.section_ID = s.section_ID
    WHERE z.Student_ID = x.Student_ID
    and semester= 'Spring' and school_year = 2022
)
```

How It Works:

- For each student x.Student_ID, we compare all Spring 2022 sections with the ones they've registered for.
- The EXCEPT returns sections the student didn't register for.
- If that result is empty (NOT EXISTS), the student must have registered for all Spring 2022 sections.

Using Aggregate Functions

```
SELECT Student_ID
FROM GradeReport g JOIN Section s ON g.section_ID = s.section_ID
WHERE semester= 'Spring' and school_year = 2022
GROUP BY Student_ID
HAVING COUNT(s.section_ID) = (SELECT COUNT(section_ID)
FROM Section
WHERE semester= 'Spring' and school_year = 2022)
```

How It Works:

- For each student, count how many Spring 2022 sections they registered for.
- Compare that count to the total number of Spring 2022 sections.
- If the numbers match, the student registered for all Spring 2022 sections.

Function

- Add a new column: Gender: char(1) into the Instructor table. Update gender value (F/M) for each instructor.
- Add a new column: birthday: datetime into the Instructor table. Update birthday value for each instructor.

Calculate the age of an instructor based on the provided date of birth

```
CREATE FUNCTION fn_GetAge(@birthday date)
RETURNS int
AS
BEGIN
    RETURN YEAR(GETDATE()) - YEAR(@birthday)
END
```

Test function

```
SELECT dbo.fn_GetAge('1988-08-06')
```

Drop function

```
DROP FUNCTION [ IF EXISTS ] dbo.fn_GetAge
```

Calculate the age of an instructor based on the provided Instructor_ID

```
GO
CREATE FUNCTION fn_GetAge(@Instructor_ID)
RETURNS int
AS
BEGIN
    RETURN (SELECT Datediff (yyyy,birthday,getdate())
            FROM Instructor
            WHERE instructor_ID = @instructor_ID)
END
GO
```

Apply function in query

Query: Please provide information about male instructors (instructor_ID, instructor_Name) who have reached retirement age (≥ 60)

```
SELECT instructor_ID, instructor_Name
FROM Instructor
WHERE Gender= 'M' AND dbo.fn_GetAge(birthday)>=60
```

Query: Please provide information about instructors (instructor_ID, instructor_Name) who have reached retirement age (Male ≥ 60 ; Female ≥ 55)

```
SELECT Instructor_id, Instructor_Name, dbo.fn_GetAge(birthday) AS Age
FROM Instructor
WHERE dbo.fn_GetAge(birthday) >=
    CASE Gender
        WHEN 'F' THEN 55
        WHEN 'M' THEN 60
        ELSE 1000 -- fallback to prevent NULL (or use 0 depending on logic)
    END
```

Stored Procedure

Write a Stored Procedure to add a new instructor

```
GO
CREATE PROCEDURE sp_AddInstructor
    @instructor_id varchar(9),
    @instructor_Name nvarchar(50),
    @phone nvarchar(9),
    @department_Id varchar(5),
    @salary int
AS
IF EXISTS(SELECT COUNT(instructor_id) FROM Instructor WHERE Department_Id =
@department_Id HAVING COUNT(instructor_id) >= 10)
BEGIN
```

```
        ;THROW 50000, 'A department must not have more than 10 employees', 1
    RETURN
END

DECLARE @head_salary int
SELECT @head_salary = salary FROM Department d JOIN Instructor I ON
d.department_id = @department_id and d.department_head = i.instructor_id
IF @head_salary < @salary
BEGIN
    ;THROW 50000, 'An instructor's salary must not greater than her/his
    department_head 's salary', 1
    RETURN
END
IF @salary <1000 OR @salary > 3000
BEGIN
    ;THROW 50000, 'Salary must be within in the range of [1000, 3000]', 1
    RETURN
END
ELSE
INSERT INTO Instructor (instructor_id, Instructor_Name, Phone, department_id,
salary) VALUES (@instructor_id, @Instructor_Name, @Phone, @department_Id,
@salary);
GO
```

Execute a procedure

```
EXEC sp_AddInstructor 'I011', 'Test Instructor','0999999999', 'CS', 1000;
```

Drop a stored procedure

```
DROP PROCEDURE [ IF EXISTS ] dbo.sp_AddInstructor
```